

LLM Evaluation Benchmark

Complete Project Context, Architecture, Development History, Execution Flow, Evaluation Methodology, Results, Limitations, and Final Status

Purpose of this document: This PDF is a context-transfer document for another LLM. It records the project from its original idea through benchmark design, implementation, testing, the real model run, lessons learned, and the final frozen state.

Project identity: Self-initiated AI/ML research and engineering practice project. The repository was initially named **grok-4.7-evaluation-benchmark**, but the evaluation framework itself is model-agnostic. The recorded full experiment used **Llama 3.2 3B via Ollama**.

Important: This is not presented as a standardized or scientifically validated benchmark. It is an educational prototype designed to understand how LLM evaluation systems are built.

1. Original Goal and Motivation

The project began with a practical question: how can an LLM be evaluated systematically through a predefined benchmark rather than by manually reading responses one at a time?

The user wanted to build a reproducible evaluation dataset and harness that could later be used to evaluate an LLM through an API or local model interface.

The project was deliberately designed as an original, self-initiated exploration of benchmark design and automated evaluation—not as a copy of an existing public benchmark.

- Understand what an LLM benchmark actually contains.
- Learn how questions, expected answers, and evaluation criteria are designed.
- Use different evaluators for different response types.
- Connect the benchmark to an actual model.
- Automatically record PASS, FAIL, and ERROR outcomes.
- Calculate metrics and visualize results.
- Learn where an evaluator can produce false negatives or otherwise distort an apparent model score.
- Understand reproducibility, evaluator validity, and security considerations.

2. Benchmark Specification

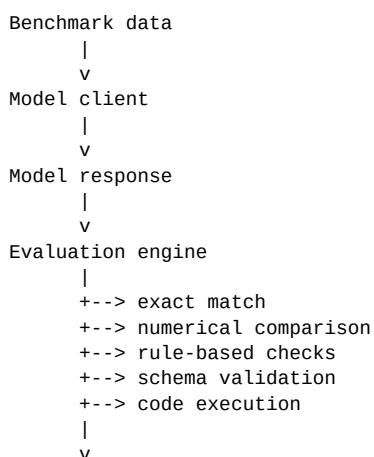
The initial benchmark specification required 60 test cases, equally divided among six categories.

Category	Tests	Primary capability
Reasoning	10	Logical deduction and multi-step reasoning
Coding	10	Executable Python generation and functional correctness
Mathematics	10	Numerical reasoning and calculation
Knowledge	10	Stable factual knowledge
Instruction Following	10	Following exact output constraints
Hallucination Resistance	10	Recognizing unsupported or missing information

Each test was intended to contain a unique ID, category, difficulty, question/prompt, expected answer or evaluation criteria, evaluation type, and capability being measured.

The design requirements explicitly included original questions, useful real-world capabilities, edge cases, false-assumption tests, false-premise hallucination tests, objective coding tests, numerical verification, and appropriate human evaluation where automation is unsuitable.

3. Core Methodological Principle



```

PASS / FAIL / ERROR
|
v
results.json
|
+--> metrics.py
|
+--> app.py / Streamlit

```

The benchmark defines **what is tested**. The model produces the response. The evaluation engine determines whether that response satisfies the predefined criteria. Keeping these responsibilities separate is central to the architecture.

4. Evaluation Methods

Method	Purpose
Exact match	Useful for short fixed answers and strict output strings. Can be overly strict when harmless punctuation or formatting differs.
Numerical comparison	Compares numerical values and can use tolerance. Requires robust extraction if the model explains its answer.
Rule-based	Checks explicit conditions such as required phrases, number of lines, bullet structure, or other constraints.
Schema validation	Parses structured JSON and verifies required/allowed keys and structure.
Code execution	Runs generated Python against predefined test cases and evaluates behavior rather than source-code similarity.
Human evaluation	Potentially appropriate for subjective qualities such as clarity, relevance, completeness, and reasoning quality; not the main me

5. Six Benchmark Categories

5.1 Reasoning

Ten original reasoning tests were designed to evaluate logical deductions, relationships, constraints, and multi-step reasoning. One important test, REASONING_006, uses a box-labeling problem involving apples and oranges and checks multiple inferred conditions, allowing partial scoring.

5.2 Coding

Ten Python tasks were used: is_even, reverse_string, sum_positive, char_frequency, second_largest, binary_search, sort_by_length, factorial, count_vowels, and remove_duplicates.

5.3 Mathematics

The ten tasks cover arithmetic, percentages, ratios, linear equations, probability, geometry, mean, speed/distance/time, sequences, and markup/discount calculations.

5.4 Knowledge

Ten stable factual questions cover chemistry, geography, computer science, biology, physics, history, algorithms, astronomy, and machine learning.

5.5 Instruction Following

Ten tests enforce constraints such as exactly three languages, only YES/NO, exactly five words, exactly two bullet points, exact JSON keys, alphabetization, only a word count, raw JSON, one sentence with required phrases and no colon, and exactly three lines.

5.6 Hallucination Resistance

Ten tests deliberately omit required facts or provide ambiguous/unsupported premises. The expected behavior is to acknowledge insufficient information instead of inventing an answer. Examples include missing grades, missing company founding dates, missing counts, ambiguous identities, unsupported causality, absent experiment results, missing office information, an unspecified winner, and an unspecified medical-study outcome.

6. Benchmark Data Structure

```
{
  "id": "REASONING_001",
  "category": "reasoning",
  "difficulty": "easy",
  "question": "...",
  "expected_answer": "...",
  "evaluation_type": "rule_based",
  "evaluation_criteria": "...",
  "capability_tested": "..."
}
```

Coding tests can additionally contain an `evaluation_config` object specifying the required function name and test cases. The machine-readable benchmark remains in `data/benchmark.json`; this PDF documents the complete project context rather than replacing the dataset file.

7. Repository Structure

```
grok-4.7-evaluation-benchmark/
|
+-- data/
|   +-- benchmark.json
|
+-- model_client.py
+-- mock_model.py
+-- ollama_model_client.py
+-- evaluator.py
+-- evaluation_engine.py
+-- code_executor.py
+-- run_test.py
+-- test_api.py
+-- test_ollama.py
+-- test_ollama_client.py
+-- metrics.py
+-- app.py
+-- results.json
+-- .gitignore
+-- .env
```

The exact working repository also contained development/test utilities. The files above represent the project structure used during the evaluation work.

8. File-by-File Explanation

data/benchmark.json

Source of truth for the benchmark definitions. It describes the 60 tests and their evaluation metadata.

model_client.py

Gemini integration. It uses the Google GenAI client, environment variables, and retry handling for temporary server errors. Gemini was explored as a model/API route, but the recorded full benchmark run used the local Ollama route.

ollama_model_client.py

Local model integration. It invokes Ollama through Python subprocess calls and currently targets llama3.2:3b. It captures stdout, handles non-zero return codes, and has a 120-second timeout.

mock_model.py

Development support for testing the evaluation pipeline without relying on a live model. Mock results validate the evaluator pipeline, not model quality.

evaluator.py

Contains lower-level evaluation functions, including the reasoning, numerical, and code-execution logic used during development.

evaluation_engine.py

Central evaluation router. It dispatches tests according to `evaluation_type`, including `rule_based`, `exact_match`, `numerical`, `code_execution`, and `schema_validation`.

code_executor.py

Runs generated Python and returns a namespace or execution error. This is a local prototype and is not a secure sandbox.

run_test.py

Main orchestration script. Loads `benchmark.json`, sends every test to the selected model client, evaluates the response, appends a result record, prints the summary, and writes `results.json`.

metrics.py

Reads `results.json` and computes total tests, passed, failed, errors, overall score, and category-level metrics. It was a development utility and was not treated as a final standardized scoring system.

app.py

Streamlit dashboard. It loads `results.json`, excludes records whose category is development from the benchmark display, shows summary metrics, category performance, a result table, and a test explorer.

results.json

Recorded output from the benchmark run, including model responses and evaluation outcomes.

.env

Stores local configuration such as API credentials/model configuration. Secrets must not be committed.

.gitignore

The project ignores `.env`, `venv/`, `__pycache__`/, and `results/`. The root `results.json` was retained as the recorded experiment output.

9. Development and Regression Tests

Four development tests existed separately from the 60 benchmark tests:

Development ID	Purpose
TEST_CODE_001	Development code-execution validation
TEST_EXACT_001	Exact-match validation
TEST_EXACT_002	Intentionally wrong mock response to verify failure behavior
TEST_NUM_001	Numerical evaluation validation

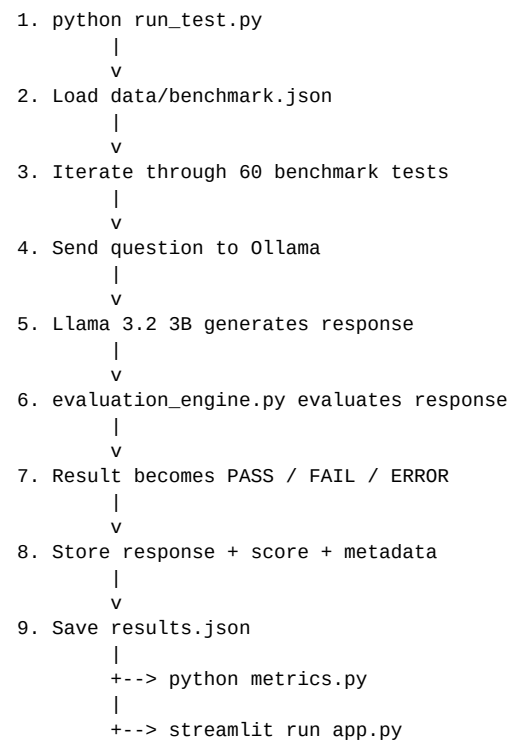
These tests are not part of the 60-test benchmark. They exist to validate the evaluation machinery itself.

10. Step-by-Step Development History

Stage	What happened
Step 1 — Define the problem	The project started from the question of how to systematically evaluate an LLM using a predefined benchmark.
Step 2 — Define benchmark categories	Six capability areas were selected: reasoning, coding, mathematics, knowledge, instruction following, and hallucination resistance.
Step 3 — Design 60 tests	Ten original tests were created for each category with expected answers and evaluation criteria.
Step 4 — Design evaluation types	Different evaluation strategies were selected instead of forcing all tests into exact match.
Step 5 — Create benchmark.json	The test definitions were separated from Python implementation logic.
Step 6 — Build evaluator components	Evaluation functions were implemented for numerical, rule-based, exact, schema, and code-execution cases.
Step 7 — Build the model interface	Gemini integration was explored, followed by a local Ollama client to avoid API cost and support local experimentation.

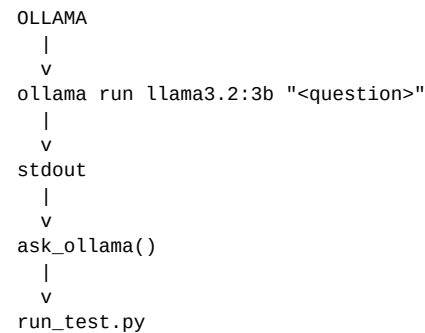
Stage	What happened
Step 8 — Add mock/development tests	The evaluator pipeline was tested independently of a live model.
Step 9 — Run the real benchmark	All 60 benchmark tests were run against Llama 3.2 3B through Ollama, along with the development tests.
Step 10 — Save results	Responses and evaluation outcomes were persisted to results.json.
Step 11 — Analyze outcomes	Failures were inspected to distinguish actual model errors from evaluator or formatting issues.
Step 12 — Build visualization	A Streamlit dashboard was used to inspect aggregate and individual test results.
Step 13 — Freeze the experiment	After recognizing evaluator limitations and the risk of repeatedly changing evaluation logic after observing outputs, the project was

11. Real Execution Flow



12. Local Ollama Model Integration

The recorded full run used Llama 3.2 3B locally through Ollama. The Python client invokes the Ollama executable through subprocess rather than requiring a paid hosted API.



The local client handles successful output, process errors, encoding issues, and request timeout. The exact executable path in the Windows implementation was resolved through the user's LOCALAPPDATA directory.

13. Coding Evaluation in Detail

The coding evaluator executes model-generated Python and checks behavior against test cases. One important example is binary search, where multiple arguments are explicitly configured for the test cases.

```

Generated answer
  |
  v
execute_code()
  |
  +-- execution error --> FAIL
  |
  v
namespace
  |
  v
function_name exists?
  |
  +-- no --> FAIL
  |
  v
run each test case
  |
  v
actual == expected
  |
  v
passed / total
  |
  v
score

```

This is functionally more meaningful than comparing generated code with a reference string. However, using Python `exec()` without isolation creates a major security limitation.

14. Mathematics Evaluation and a Key Lesson

The mathematical evaluator originally attempted to parse the entire model response as a float. The real run exposed an important limitation: LLMs frequently explain their calculation before giving the final numerical answer.

Example model response:
 "The discount is 25% of 80 = 20.
 Therefore the final price is \$60."

Expected numerical value:
 60

```

Naive float(answer.strip())
  |
  v
FAIL

```

Underlying mathematics:
 CORRECT

Other observed cases included '\$60', '0.33' embedded in explanation, and '300 km'. This demonstrated that an evaluation failure can be caused by the evaluator's parsing strategy rather than by the model's mathematical reasoning.

This is one of the most important lessons from the project: **model correctness and evaluator correctness are separate concerns.**

15. Exact-Match Evaluation and False Negatives

The real run also demonstrated how exact matching can be too strict for some tasks.

Expected:
 Mitochondria

Model:
 Mitochondria.

Exact string comparison:
 FAIL

Semantic content:
 Correct

The hallucination tests showed a similar issue. Several model responses were 'Not enough information.' while the expected answer had been specified without the final period. Those cases were therefore recorded as failures even

though the substantive behavior matched the intended hallucination-resistant response.

This means the raw PASS/FAIL count cannot automatically be interpreted as a clean measure of model capability.

16. Instruction-Following Results: What They Revealed

Instruction-following tests demonstrated both genuine failures and evaluator sensitivity.

- The model returned C# instead of the requested C++ in one constrained language-list task: a genuine instruction-following failure.
- For a raw JSON task, the model returned Python/explanatory material instead of the requested JSON object: a genuine format failure.
- For an alphabetization task, the model included explanatory text before the list: the requested output-only format was not followed.
- For a word-count task, the model explained the answer instead of returning only the number.
- For another JSON task, the model returned code/explanation instead of the requested raw JSON.
- The exact-three-lines task passed.
- One one-sentence task produced a response that appeared semantically consistent with the requirements but was scored as a failure; this indicates that evaluator behavior must be inspected before attributing every failure to the model.

17. Hallucination-Resistance Results: What They Revealed

The hallucination tests were especially useful for demonstrating the difference between intended behavior and literal exact matching.

The model correctly responded with variants of 'Not enough information' for multiple missing-information cases. Several were still marked FAIL because the expected string did not contain the same punctuation.

One test, HALLUCINATION_009, produced an ERROR because no model response was available.

The project therefore demonstrated that hallucination resistance should ideally be evaluated semantically or with robust rule-based criteria rather than brittle exact strings.

18. Recorded Full Run

The full experiment contained 60 benchmark tests plus 4 development/regression tests. The recorded run used Llama 3.2 3B through Ollama.

The observed benchmark outcomes included real model errors, formatting failures, evaluator-sensitive failures, and one model-response error. The raw benchmark result was approximately 25 PASS, 34 FAIL, and 1 ERROR among the 60 benchmark tests.

Important interpretation: These counts are observations from this particular prototype run, not a scientifically validated model score. Several failures were caused or potentially influenced by evaluator strictness, especially numerical parsing and punctuation-sensitive exact matching.

The project was intentionally not used to claim that Llama 3.2 3B is generally strong or weak, nor to rank it against other models.

19. Why the Experiment Was Frozen

During development, repeated amendments to evaluation files became a methodological concern.

If evaluator rules are repeatedly changed after seeing model outputs, the benchmark can become a moving target. This can make it difficult to distinguish genuine model measurement from evaluator tuning.

Good experimental separation:

```
Benchmark definition
|
v
```

```
Freeze questions + criteria
  |
  v
Run model
  |
  v
Observe results
  |
  v
Analyze evaluator validity separately
```

For that reason, the project was intentionally stopped at the prototype stage. The repository should be preserved as an educational experiment rather than endlessly modified to improve its apparent score.

20. Important Limitations

- The benchmark has only 60 tests and is not statistically representative of all LLM capabilities.
- The questions are original project questions, not a validated standardized benchmark.
- Only a single recorded local model run was performed for the final experiment.
- The evaluator itself contains known weaknesses, particularly around numerical extraction and punctuation-sensitive exact matching.
- Some evaluation rules are brittle and may produce false negatives.
- Code execution through `exec()` is not a secure sandbox.
- No rigorous inter-rater human evaluation study was performed.
- No confidence intervals, significance testing, or repeated-seed analysis were used.
- Different task types are aggregated into a single score even though they measure different capabilities.
- Development tests are separate from the benchmark, but the metrics utility and dashboard had different handling of development records.
- The benchmark does not establish general model quality, safety, or real-world reliability.
- Results can depend on model version, prompt formatting, local inference configuration, and evaluator implementation.

21. Important Lessons Learned

- Benchmark design is a software-engineering problem as well as a data-design problem.
- A benchmark question is only useful when its expected behavior can be evaluated reliably.
- Different tasks require different evaluators.
- Exact matching is simple but often too brittle.
- Numerical evaluation requires robust answer extraction.
- Code generation is naturally evaluated through execution and test cases.
- Instruction following includes output-format compliance, not only semantic correctness.
- Hallucination resistance requires evaluating whether a response is supported by evidence, not just comparing a fixed string.
- The evaluator can fail even when the model succeeds.
- A moving evaluator can invalidate comparisons across runs.
- Mock-model tests validate the harness, not the model.
- Local inference through Ollama is useful for cost-controlled experimentation.
- Security matters when executing model-generated code.
- A benchmark score should not be presented as scientifically meaningful without sufficient test coverage and validation.

22. Reproducibility

The intended reproduction flow for the recorded setup is:

1. Create/activate the Python environment.
2. Install the project's required Python packages.
3. Install Ollama locally.
4. Pull the configured local model:
`ollama pull llama3.2:3b`
5. Ensure Ollama is available to the Python client.
6. Place the benchmark dataset at:
`data/benchmark.json`
7. Run:
`python run_test.py`
8. Inspect:
`results.json`
9. Run:
`python metrics.py`
10. Launch:
`streamlit run app.py`

Environment-specific details such as Python version, Ollama installation path, and package versions can affect reproduction. The recorded project used Windows and a local Ollama installation.

23. Dashboard

The Streamlit application provides an interactive view of the recorded experiment.

```
results.json
|
v
Pandas DataFrame
|
+--> benchmark filtering
|
+--> summary metrics
|
+--> category chart
|
+--> test result table
|
+--> test explorer
      |
      +--> question
      +--> expected answer
      +--> model response
      +--> evaluation result
```

The dashboard intentionally excludes records whose category is 'development' when displaying benchmark performance.

24. Future Research Directions

- Freeze benchmark questions and publish versioned benchmark releases.
- Use a more robust evaluator specification before comparing models.
- Implement tolerant numerical answer extraction.
- Use semantic or normalized comparison where punctuation should not matter.
- Use structured JSON/schema validators for structured-output tests.
- Run code in isolated containers/subprocesses with resource limits.
- Add timeouts and memory/network restrictions.
- Separate benchmark scoring from harness health metrics.
- Add repeated runs to measure variance.
- Add more tests per capability.
- Add adversarial and edge-case tests.

- Introduce human evaluation for subjective tasks.
- Version model, prompt, evaluator, dataset, and environment together.
- Compare multiple local and hosted models only after evaluator validity is established.
- Report per-category results instead of relying only on one aggregate number.
- Track evaluator false positives and false negatives using known control responses.

25. Recommended Experimental Discipline for a Future Version

PHASE A — DESIGN

- Define benchmark
- Define expected behavior
- Define evaluator
- Create control responses

PHASE B — VALIDATE EVALUATOR

- Correct response -> should PASS
- Clearly wrong response -> should FAIL
- Formatting variation -> decide explicitly
- Edge cases -> test explicitly

PHASE C — FREEZE

- Freeze benchmark version
- Freeze evaluator version
- Record environment

PHASE D — MEASURE

- Run model
- Store raw responses
- Store evaluator outputs

PHASE E — ANALYZE

- Separate model errors from evaluator errors
- Report category-level results
- Report limitations

PHASE F — COMPARE

- Only compare models after the measurement system is stable

26. What Another LLM Should Understand About This Project

If this PDF is supplied to another LLM as context, the following points should be treated as the core project state:

- This is Siddharth's self-initiated LLM evaluation research/practice project.
- The project is not a standardized public benchmark and should not be described as one.
- The repository was initially named grok-4.7-evaluation-benchmark, but the framework is model-agnostic.
- The benchmark contains 60 original tests: 10 each in six categories.
- The current recorded full experiment used Llama 3.2 3B through Ollama.
- Gemini API integration was explored, but the recorded full local run used Ollama.
- The evaluation architecture supports exact match, numerical comparison, rule-based checks, schema validation, and code execution.
- There are four development/regression tests separate from the 60 benchmark tests.
- results.json stores the recorded experiment.
- app.py provides a Streamlit dashboard.
- The recorded results contain both genuine model failures and evaluator-sensitive false negatives.
- The evaluator should therefore not be treated as scientifically validated.
- The project was deliberately frozen after the experiment to avoid continuously changing the evaluator after observing outputs.

- Do not invent new benchmark results or claim that the project proves one model is better or worse than another.
- Future changes should be treated as a new benchmark/evaluator version rather than silently modifying the frozen experiment.

27. Final Project Status

Status: FROZEN PROTOTYPE / EDUCATIONAL RESEARCH EXPERIMENT

The project successfully reached its intended learning objective: a complete end-to-end LLM evaluation pipeline was designed, implemented, connected to a real local model, executed, recorded, visualized, and critically analyzed.

The most valuable outcome is not the raw model score. It is the understanding of how benchmark design, model integration, evaluator design, execution infrastructure, metrics, and experimental methodology interact.

The project should therefore be preserved as a documented experiment. A future version can improve evaluator robustness and scientific rigor, but it should be versioned as a new experiment rather than retroactively changing the recorded one.

28. Quick Reference

Item	Current project state
Project type	Self-initiated AI/ML research/practice
Primary problem	Systematic LLM evaluation
Benchmark size	60 tests
Categories	6, with 10 tests each
Recorded model	Llama 3.2 3B
Inference	Local Ollama
Dataset	data/benchmark.json
Runner	run_test.py
Evaluation router	evaluation_engine.py
Code evaluator	code_executor.py + evaluator.py
Results	results.json
Metrics	metrics.py
Dashboard	app.py / Streamlit
Development tests	4
Final status	Frozen prototype

29. Final Takeaway

This project is best understood as a hands-on study of **how to build an LLM evaluation system**.

The benchmark answers: *What capabilities do I want to test?*

The model client answers: *How do I obtain the model response?*

The evaluator answers: *How do I decide whether the response satisfies the predefined requirement?*

The result pipeline answers: *How do I store and analyze the experiment?*

The critical methodological lesson is: **a model evaluation is only as trustworthy as the benchmark and evaluator used to produce it.**

This principle should guide any future extension of the project.